

SEMESTER 1 PORTFOLIO

Computer Science Student Portfolio

ABDULRAHMAN ALGAILANI

Universiti Sains Malaysia (USM) | School of Computer Sciences

Email: abdulrahmaanalgailani@gmail.com

Academic Status	Completed Semester 1 and passed all courses
Portfolio Focus	Programming, discrete mathematics, computer organization, academic development
Courses Included	Principles of Programming, Discrete Structure, Computer Organization, Philosophy, Malay

A clean, evidence-based portfolio showing first-semester learning progress and practical work.

1. About Me / Student Profile

Student academic profile and first-semester learning overview.

I am ABDULRAHMAN ALGAILANI, a Computer Science student at Universiti Sains Malaysia (USM). **I have completed Semester 1 and passed all my courses.** During the semester, I enjoyed all five courses because each one helped me build a different part of my foundation as a computer science student.

Program Area	Computer Science
University	Universiti Sains Malaysia (USM)
Email	abdulrahmaanalgailani@gmail.com
Semester Completed	Semester 1
Academic Result Statement	Passed all Semester 1 courses

SEMESTER 1 COURSES

- Principles of Programming - learned basic programming logic, variables, input/output, conditions, loops, arrays, and problem solving.
- Discrete Structure - practiced sets, functions, recursion, algorithms, summations, matrices, recurrence relations, and mathematical induction.
- Computer Organization - studied CPU, control unit, memory hierarchy, system bus, I/O modules, instruction cycle, and interfacing concepts.
- Philosophy - improved critical thinking, reflection, ethical awareness, and reasoning skills.
- Malay - developed basic Malay communication and adaptation to studying in Malaysia.

2. Skills Developed in Semester 1

Skills are shown with evidence from programming samples, coursework, and assignments.

Skill Area	Evidence / Example	Current Level
Programming basics	C++ input/output, variables, arithmetic, if-else, loops, arrays, and validation.	Beginner - improving
Problem solving	Solved tutorial questions and transformed requirements into simple programs.	Beginner - improving
Discrete mathematics	Sets, power sets, functions, recursive definitions, matrices, algorithms, and induction.	Foundation built
Computer systems understanding	CPU, control unit, memory hierarchy, I/O modules, system bus, and instruction cycle.	Foundation built
Documentation	Portfolio organization, project descriptions, evidence pages, and reflections.	Improving
Communication	Malay learning, academic writing, and explaining technical work in simple English.	Improving

TOOLS AND CONCEPTS

- Languages: C++ basics, Markdown for GitHub README files.
- Concepts: variables, control structures, arrays, validation, sets, matrices, induction, CPU organization, memory hierarchy.
- Study skills: handwritten coursework, tutorial practice, poster preparation, reflection, and documentation.

3. Project 1 - Programming Sample Tutorial Questions

Course link: Principles of Programming

PROJECT DESCRIPTION

This project collects beginner C++ practice programs completed during Semester 1. The samples show how simple real-world problems can be solved using programming fundamentals. The two included samples are **Calculate Age from IC Number** and **Salsa Sales Array Report**.

Sample	Main Idea	Programming Concepts
Calculate Age from IC Number	Extract birth year from an IC number and calculate age.	Input, arithmetic, type conversion, if-else decisions
Salsa Sales Array Report	Record jars sold for five salsa types and report total, highest, and lowest sales.	Arrays, loops, validation, accumulation, comparison

LEARNING OUTCOMES

- Understood how to read user input and display clear output.
- Practiced using conditions to choose different program paths.
- Used loops to repeat tasks and reduce duplicated code.
- Used arrays to store related data and process it efficiently.
- Learned that code should be tested and improved after it works.

Project 1 Evidence A - Calculate Age from IC Number

Programming sample focusing on variables, arithmetic, and conditional logic.

Programming Sample: Calculate Age from IC Number

```
01 #include <iostream>
02 using namespace std;
03
04 int main() {
05     double ic;
06     cout << "Enter your IC number: ";
07     cin >> ic;
08     int year, birthYear, age;
09
10     year = ic / 10000000000; //extract YY only
11     birthYear;
12
13     if ( year <= 24)
14         birthYear = 2000 + year; // for 2000-2024
15     else
16         birthYear = 1900 + year; // for 1900-1999
17
18     age = 2025 - birthYear;
19
20     cout << "Your birth year is: " << birthYear << "\n";
21     cout << "Your age is: " << age << "\n";
22
23     return 0;
24 }
```

Evidence: C++ sample for calculating birth year and age from an IC number.

WHAT THIS SAMPLE DEMONSTRATES

- Reads an IC number from the user.
- Extracts the first two digits to estimate the birth year.
- Uses if-else logic to decide whether the year belongs to the 1900s or 2000s.
- Calculates the age using the selected birth year and the year 2025.

Project 1 Evidence B - Salsa Sales Array Report

Programming sample focusing on arrays, loops, validation, and simple reporting.

Programming Sample: Salsa Sales Array Report

```

01 #include <iostream>
02 #include <string>
03 using namespace std;
04
05 int main() {
06     const int SIZE = 5;
07     string salsa[SIZE] = {"mild", "medium", "sweet", "hot", "zesty"};
08     int jars[SIZE];
09
10     for (int i = 0; i < SIZE; i++) {
11         do {
12             cout << "Enter jars sold for " << salsa[i] << ": ";
13             cin >> jars[i];
14             if (jars[i] < 0) cout << "Invalid! Cannot be negative.\n";
15         } while (jars[i] < 0);
16     }
17
18     int total = 0;
19     int highestIdx = 0, lowestIdx = 0;
20
21     for (int i = 0; i < SIZE; i++) {
22         total += jars[i];
23         if (jars[i] > jars[highestIdx]) highestIdx = i;
24         if (jars[i] < jars[lowestIdx]) lowestIdx = i;
25     }
26
27     cout << "\n--- Salsa Sales Report ---\n";
28     for (int i = 0; i < SIZE; i++)
29         cout << salsa[i] << ": " << jars[i] << endl;
30
31     cout << "Total sales: " << total << endl;
32     cout << "Highest selling: " << salsa[highestIdx] << endl;
33     cout << "Lowest selling: " << salsa[lowestIdx] << endl;
34
35     return 0;
36 }

```

Evidence: C++ array program that records sales and calculates summary statistics.

WHY THIS IS STRONGER THAN A VERY SIMPLE PROGRAM

- It uses two related arrays: one for salsa names and one for jar counts.
- It validates input by rejecting negative sales values.
- It calculates total sales by accumulating values in a loop.
- It finds the highest and lowest selling salsa using index comparison.
- It prints a structured report, which is closer to real programming output.

Project 1 Reflection - Programming Growth

How the programming samples helped build confidence.

Before Practice	After Practice
I mainly understood code line by line.	I started understanding how to plan input, processing, and output.
Loops and arrays were new concepts.	I can use loops and arrays for repeated data processing.
I focused only on making code run.	I now understand the importance of validation and readable output.
I did not think much about improvement.	I can now identify ways to improve code, such as using better data types and functions.

NEXT PROGRAMMING GOALS

1. Rewrite the IC program using string input and better date handling.
2. Convert repeated logic into functions.
3. Add comments explaining each part of the code.
4. Create clear project documentation for each program.
5. Practice more problems involving arrays, strings, and functions.

4. Project 2 - Discrete Mathematics Coursework / Assignments

Course link: Discrete Structure

PROJECT DESCRIPTION

This project presents selected evidence from Discrete Structure coursework. The coursework included Part A and Part B questions and required handwritten solutions. It helped me practice mathematical reasoning, notation, and structured problem solving.

Topic Area	Examples Practiced
Set theory	Subsets, power sets, set operations, complements, union, and intersection.
Functions	Function composition, inverse function thinking, and preimage calculation.
Recursion and algorithms	Recursive definitions, algorithm design, and recursive GCD ideas.
Summations and matrices	Sigma notation, matrix addition, and matrix multiplication.
Proof and induction	Base case, inductive hypothesis, inductive step, and proof conclusion.

LEARNING OUTCOMES

- Improved mathematical notation and attention to detail.
- Connected programming logic with mathematical logic.
- Practiced proof writing through induction and recursive reasoning.
- Built a foundation for algorithms, data structures, and computer science theory.

Project 2 Evidence - Discrete Structure Coursework

Selected pages from the coursework evidence, excluding cover details.



USM APEX
School of Computer Sciences

CKC111 - DISCRETE STRUCTURES

SEMESTER 1 2025/2026

TUTORIAL 2 (COURSEWORK) - Individual

This tutorial consists of two parts, **Part A** (10 questions) and **Part B** (8 questions). Please answer all questions. Kindly provide the **handwritten solution**, not the typed version. Please scan your answer scripts using a scanner or a mobile scanning app, and upload them to e-Learn.

Due date (softcopy): 29 Dec 25 (softcopy, before 12.00 pm) @ e-Learn.

PART A:

- Let $A = \{3, 7, 8, 16, 17, 28\}$, $B = \{16, 28, 29\}$, $C = \{7, 8, 16\}$, and $D = \{28, 29\}$. State whether the following are true or false.
 - $C \subset A$ True (Every element of C is in A)
 - $D \subset A$ False (28 is in D but not in A)
 - $B \subset A$ True (28 is in B but not in A , so B is not subset of A)
- List out all the power set of:
 - $A = \{0, b, \{b\}\}$ $P(A) = \{\emptyset, \{0\}, \{b\}, \{0, b\}, \{\{b\}\}, \{0, \{b\}\}, \{b, \{b\}\}, \{0, b, \{b\}\}$
 - $B = \{0, \{0\}, \{0, \{0\}\}\}$ $P(B) = \{\emptyset, \{0\}, \{\{0\}\}, \{0, \{0\}\}, \{\{0, \{0\}\}\}, \{0, \{0, \{0\}\}\}, \{\{0\}, \{0, \{0\}\}\}, \{0, \{0, \{0, \{0\}\}\}\}$
- Let $U = \{A, B, C, D, E, F, G, H, I, J, K\}$, $X = \{B, D, F, H, J, K\}$, $Y = \{A, B, C, D, E, F, K\}$ and $Z = \{D, E, F, G, H, I, J\}$. Find:
 - $X \cup Y \cap Z = \{B, D, F, H, J, K\} \cup \{A, B, C, D, E, F, K\} \cap \{D, E, F, G, H, I, J\} = \{D, E, F, H, J, K\}$
 - $X \cap Y \cap Z = \{B, D, F, H, J, K\} \cap \{A, B, C, D, E, F, K\} \cap \{D, E, F, G, H, I, J\} = \{D, F, H, J\}$
 - $X \cap Z \cap Y = \{B, D, F, H, J, K\} \cap \{D, E, F, G, H, I, J\} \cap \{A, B, C, D, E, F, K\} = \{D, F, H, J\}$
 - $X \cap Y \cup Z = \{B, D, F, H, J, K\} \cap \{A, B, C, D, E, F, K\} \cup \{D, E, F, G, H, I, J\} = \{B, D, F, H, J, K\} \cup \{D, E, F, G, H, I, J\} = \{B, D, E, F, G, H, I, J, K\}$
 - $X \cap Y \cup Z - X \cap Z \cap Y = \{B, D, F, H, J, K\} \cap \{A, B, C, D, E, F, K\} \cup \{D, E, F, G, H, I, J\} - \{D, E, F, H, J\} = \{B, K\}$

PART B:

- Calculate the following:
 - $\sum_{k=1}^5 (3^{2k} + 2^k) = 3^2 + 2^1 + 3^4 + 2^2 + 3^6 + 2^3 + 3^8 + 2^4 + 3^{10} + 2^5 = 666491$
 - The value of the sum, where $S = \{1, 2, 4\}$.
 $\sum_{j \in S} 5j = 5(1) + 5(2) + 5(4) = 35$
- Let $A = \begin{bmatrix} 1 & 4 & 0 & -9 \\ 3 & 6 & 2 & -3 \\ 1 & -7 & 8 & 0 \end{bmatrix}$, $B = \begin{bmatrix} 0 & 8 \\ 1 & 7 \\ 3 & 0 \\ 4 & 2 \end{bmatrix}$, $C = \begin{bmatrix} 4 & 41 & 0 & 2 \\ 0 & -3 & 6 & 3 \\ 2 & -2 & -6 & -1 \end{bmatrix}$.
 - Find $A + C$
 - Find product AB
- A company earns a total profit of 20 billion in 2020. The company's profit increases at a rate of 9% annually.
 - Set up a recurrence relation for the profit of the company n years after 2020. $a_n = 1.09^n a_0$, $a_0 = 20$
 - Find an explicit formula for the profit of the company n years after 2020. $a_n = 20(1.09)^n$
 - What is the company's profit in 2029? $a_9 = 20(1.09)^9 \approx 45.44$
- Let $P(n)$ be the statement that $1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$, for all positive integer, n .
 - What is the statement $P(1)$? $1^2 = \frac{1(1+1)(2(1)+1)}{6} = 1$
 - Show that $P(1)$ is true, completing the basis step of the proof.
 - What is the inductive hypothesis? $\text{Assume } P(k) \text{ is true for some } k \in \mathbb{N}$. $1^2 + 2^2 + \dots + k^2 = \frac{k(k+1)(2k+1)}{6}$
 - What do you need to prove in the inductive step? $P(k+1)$ is true.
 - Complete the inductive step, identifying where you use the inductive hypothesis.
 - Explain why these steps show that this formula is true whenever n is a positive integer.

Evidence: Discrete Structure coursework pages showing tutorial questions and handwritten solutions.

EVIDENCE SUMMARY

- The coursework required solving multiple questions across two parts.
- Part A included set operations, functions, recursive definitions, and algorithms.
- Part B included summations, matrices, recurrence relations, and mathematical induction.
- Handwritten work shows the process of solving, not only the final answer.

Project 2 Reflection - Mathematical Thinking

What I learned from Discrete Structure coursework.

Coursework Skill	How it helped me
Set operations	Helped me understand grouping, conditions, and relationships between data.
Functions and composition	Improved my understanding of input-output relationships.
Recursive definitions	Prepared me to understand recursive algorithms in programming.
Matrices	Introduced structured numerical data and operations.
Induction proofs	Trained me to prove that a rule works for all valid cases.

IMPROVEMENT PLAN

- Rewrite selected solutions neatly in digital format after marking is complete.
- Create summary notes for each topic with definitions and examples.
- Practice proof-writing steps: base case, hypothesis, inductive step, conclusion.
- Connect each mathematical topic to a programming example.

5. Project 3 - Computer Organisation Assignment

Course link: Computer Organization

PROJECT DESCRIPTION

This project presents a Computer Organization mini poster assignment. The poster summarizes important computer system components including CPU, control unit, interfacing and communication, memory, system-level organization, I/O modules, the instruction cycle, and the 3-bus model.

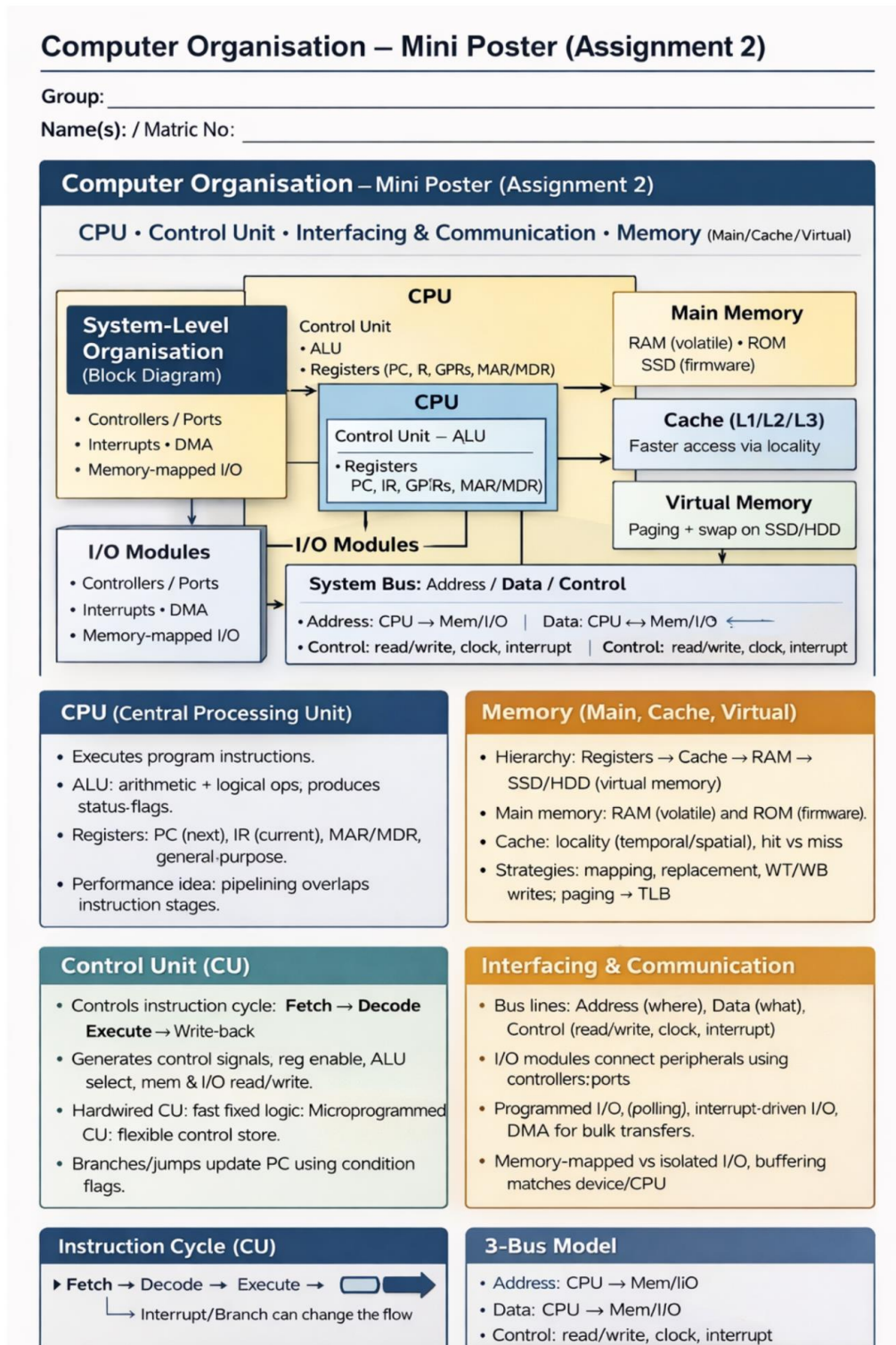
Concept	Meaning in simple words
CPU	Executes program instructions and coordinates processing.
Control Unit	Controls the instruction cycle and generates control signals.
Memory	Stores data and instructions using registers, cache, RAM, and storage.
I/O Modules	Connect the computer to external devices and peripherals.
System Bus	Transfers address, data, and control signals between components.
Instruction Cycle	Fetch, decode, execute, and write-back process.

LEARNING OUTCOMES

- Understood the main parts of a computer system.
- Learned how the CPU, memory, and I/O modules communicate.
- Connected hardware organization with how programs run.
- Practiced summarizing technical information visually in poster format.

Project 3 Evidence - Computer Organisation Mini Poster

Visual evidence from the submitted mini poster assignment.



Evidence: Computer Organization mini poster showing CPU, memory, bus, I/O, instruction cycle, and 3-bus model.

Project 3 Reflection - Understanding Computer Systems

How the assignment improved my understanding of hardware fundamentals.

Before the assignment	After the assignment
I knew computers run programs, but not the detailed internal process.	I can explain the basic roles of CPU, memory, buses, and I/O modules.
The control unit was unclear to me.	I understand that the control unit guides the instruction cycle and control signals.
Memory was mostly just RAM in my mind.	I now understand hierarchy: registers, cache, RAM, and storage/virtual memory.
I did not connect hardware to programming.	I can see how programs depend on CPU execution, memory access, and data movement.

FUTURE TECHNICAL GOALS

- Learn binary number systems and logic gates more deeply.
- Practice explaining the fetch-decode-execute cycle with simple diagrams.
- Connect Computer Organization topics with C++ memory and program execution.
- Build a GitHub notes repository for CPU and memory concepts.

6. Activities, Competitions, Clubs, and Volunteering

Honest portfolio section with completed activities and space for future additions.

Category	Completed / Evidence	Future Plan
Academic activities	Completed programming practice, discrete coursework, and computer organization poster.	Continue organizing academic evidence.
Tutorial participation	Completed tutorial questions and coursework practice.	Prepare cleaner digital notes after each topic.
Clubs	To be added when joined.	Join a computing, coding, robotics, or entrepreneurship club at USM.
Competitions	To be added when completed.	Try beginner coding challenges or hackathon-style events.
Volunteering	To be added when completed.	Look for university volunteering or student support activities.

RECOMMENDED ACTIVITIES FOR SEMESTER 2

My Goals for Semester 2

- **Enhance my programming and problem-solving skills** through consistent coding practice, personal projects, and building my GitHub portfolio.
- **Improve my academic performance** by strengthening study habits and aiming for a higher GPA.
- **Develop teamwork, discipline, and personal growth** through active participation in basketball, sports clubs, and computer science communities.

7. Final Reflection and Future Goals

Overall reflection after completing Semester 1.

Semester 1 helped me build the foundation needed for Computer Science. I liked all my courses because each course supported a different part of my development. Programming taught me how to solve problems using code. Discrete Structure trained my logical and mathematical thinking. Computer Organization helped me understand how computers work internally. Philosophy improved my reflection and critical thinking. Malay helped me communicate and adapt better to studying in Malaysia.

Goal Area	Semester 2 Plan
Programming	Write cleaner C++ programs using functions, arrays, strings, and comments.
Mathematics	Improve proof writing, recursion understanding, and problem-solving speed.
Computer systems	Understand memory, CPU execution, and low-level concepts more deeply.
Documentation	Prepare projects with explanations, screenshots, and reflections.
Academic growth	Keep a weekly learning journal and prepare early for assignments.

PORTFOLIO CLOSING STATEMENT

This portfolio represents my first step as a Computer Science student. It shows my learning progress, completed coursework evidence, and commitment to improving my technical, mathematical, and communication skills.